1. Aim

To implement a single-layer perceptron from scratch using NumPy. train it on the AND gate dataset and visualize the decision boundary.

2. Theory

A perceptron is the simplest form of artificial neural network used for binary classification. It uses a linear decision boundary to separate the classes.

For input vector $\mathbf{x} = [x_1, x_2]$, the perceptron computes:

$$z = w_1 x_1 + w_2 x_2 + b$$

$$\text{Output} \quad y = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

Weights ($\mathbf{w}$) and bias (b) are updated using the perceptron learning rule:

$$w_{new} = w_{old} + \eta\,(t - y)x \qquad b_{new} = b_{old} + \eta\,(t - y)$$

where:

- η = learning rate
- t = true target
- y = predicted output

3. Dataset (AND Gate)

x_1	x_2	Target (y)
0	0	0
0	1	0
1	0	0
1	1	1

AND Gate is linearly separable so perceptron can learn it.

4. Step by Step Explanation of Code

4.1 Import Libraries

- numpy → for numerical operations
- matplotlib.pyplot → for visualization

4.2 Perceptron Class

The class contains all the functions needed for perceptron.

4.3 __init__(self, learning_rate, n_iters)

Initializes learning rate (η), number of iterations (epochs), weights (set later), and bias (set to 0).

4.4 _step_function(self, x)

Implements the Heaviside step function.

- If $x \geq 0$ → output 1
- If $x < 0$ → output 0

4.5 fit(self, X, y)

1. Get number of samples and features.
2. Initialize weights to zeros.
3. For each epoch (n_iters):
   - For each training sample:
     a. Compute $z = w \cdot x + b$ (linear output).
     b. Apply step function → get predicted output.
     c. Compute error = target − predicted.
     d. Update weights and bias using perceptron rule.
4. Repeat until all epochs complete.

4.6 predict(self, X)

Computes $z = w \cdot x + b$ for new data and returns class (0 or 1).

4.7 Main Program

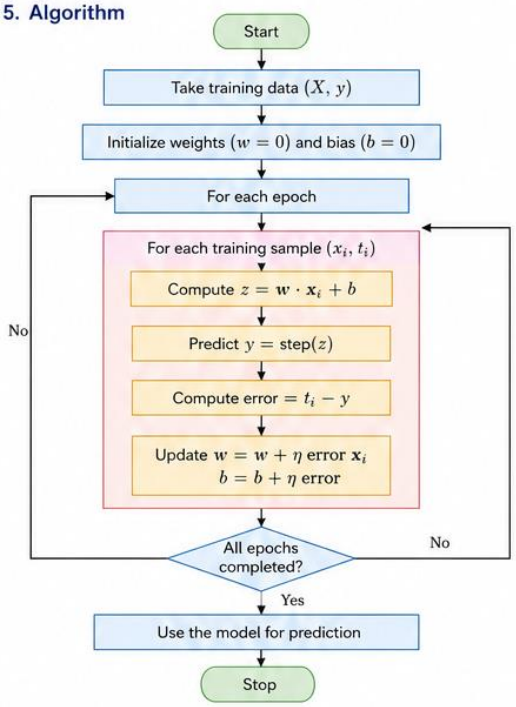
1. Create AND gate dataset.
2. Create perceptron object with learning_rate = 0.1 and n_iters = 100.
3. Train the perceptron using fit().
4. Predict outputs for the training data.
5. Print results.
6. Plot data points of both classes.
7. Calculate and draw the decision boundary line $w_1 x_1 + w_2 x_2 + b = 0$.

8. How It Works (Example)

Input (x_1, x_2)	Target (t)	Initial Output	Error ($t - y$)	Weight Update
(0, 0)	0	1 (since $0 \geq 0$)	-1	No change ($x = 0$)
(0, 1)	0	0	0	No change
(1, 0)	0	0	0	No change
(1, 1)	1	0	1	$w = [0.1, 0.1]$, $b = 0.1$

After a few epochs, weights and bias converge to values that correctly classify all points.

5. Algorithm



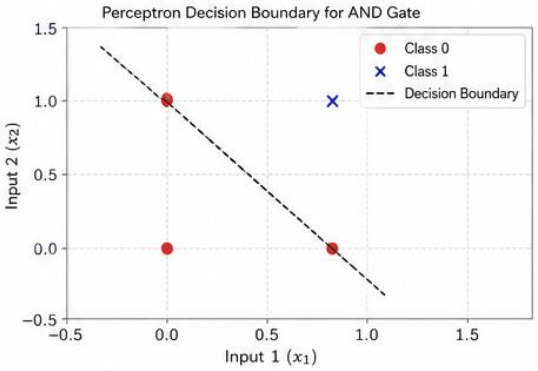
6. Sample Output

```
Input data:
[[0 0]
 [0 1]
 [1 0]
 [1 1]]
True labels:   [0 0 0 1]
Predictions:   [0 0 0 1]

Final weights: [0.1 0.1]
Final bias:    -0.1
```

The predictions match the true labels, so the perceptron has learned the AND gate correctly.

7. Visualization (Decision Boundary)



9. Result

The perceptron successfully learns the AND gate.
Final weights and bias form a linear decision boundary that separates class 1 (only point $(1, 1)$) from class 0.

10. Conclusion

We implemented a single-layer perceptron from scratch using NumPy. The model was trained on the AND gate dataset and it correctly classified all the patterns. The decision boundary is a straight line that separates the classes, confirming the working of the perceptron learning algorithm.